Lab Handout 4-E

Multi-Layer Perceptron with Early Stopping and Calibration

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Implement, train, and evaluate a small fully-connected neural network ($F \to 64 \to 32 \to C$, where F is the per-task feature count and C is the class count) in `PyTorch` for each of the four classification tasks (SF, modulation, emitter ID, packet type). Use early stopping on the validation loss, apply dropout for regularisation, and compare the MLP's test accuracy against the random-forest and SVM benchmarks from LH4-C/D. The MLP represents the first "deep" model in the project and its job is to demonstrate whether added representational capacity helps on this tabular feature set.

2 Prerequisites

- Lab Handout 4-D complete (`baselines_manifest.json` and `svm_manifest.json` present in `Code/AIML/`).
- `PyTorch` $\geq$ `2.0` installed. GPU is optional — the model is small enough to train on CPU in seconds per task.

3 Architecture

Layer	Shape / Parameters
Input	$B \times F$ (F = number of features for the task, typically 6–7)
Linear + ReLU	$F \to 64$
Dropout	$p = 0.3$
Linear + ReLU	$64 \to 32$
Dropout	$p = 0.3$
Linear	$32 \to C$ (number of classes)
LogSoftmax	applied by `CrossEntropyLoss` internally

Table 1: MLP architecture. Total parameter count is well under 10,000 per task — small enough that overfitting is controlled by dropout and early stopping rather than depth/width.

4 Procedure

4.1 Step 1 — Logging, model, and training loop

`Code/AIML/mlp_train.py` follows the same logging pattern as LH4-A through LH4-D. Single-class tasks (`mod` in the current LoRa-only dataset) are skipped because softmax + cross-entropy reduces to a constant for one class.

```python
import json
import logging
import sys
import time
from datetime import datetime, timezone
from math import sqrt
from pathlib import Path

import joblib
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from scipy.stats import norm
from sklearn.calibration import calibration_curve
from torch.utils.data import DataLoader, TensorDataset

SCRIPT_DIR    = Path(__file__).parent
LOG_DIR       = SCRIPT_DIR / "Logs"
ARTIFACTS_DIR = SCRIPT_DIR / "artifacts"
FIGS_DIR      = SCRIPT_DIR / "figs"
MANIFEST_PATH = SCRIPT_DIR / "mlp_manifest.json"

LOG_DIR.mkdir(parents=True, exist_ok=True)
FIGS_DIR.mkdir(parents=True, exist_ok=True)

run_stamp   = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
log_archive = LOG_DIR / f"mlp_train_{run_stamp}.log"
log_latest  = LOG_DIR / "mlp_train_latest.log"

log = logging.getLogger("mlp_train")
log.setLevel(logging.DEBUG)
log.handlers.clear()
fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(message)s",
                        datefmt="%Y-%m-%d %H:%M:%S")
for h in (logging.StreamHandler(sys.stdout),
          logging.FileHandler(log_archive, mode="w", encoding="utf-8"),
          logging.FileHandler(log_latest,  mode="w", encoding="utf-8")):
    h.setFormatter(fmt)
    log.addHandler(h)

log.info("=== mlp_train.py starting ===")
log.info(f"Run stamp: {run_stamp}")


class MLP(nn.Module):
```

```python
    def __init__(self, n_features, n_classes, hidden=(64, 32), p=0.3):
        super().__init__()
        layers = []; prev = n_features
        for h in hidden:
            layers += [nn.Linear(prev, h), nn.ReLU(), nn.Dropout(p)]
            prev = h
        layers.append(nn.Linear(prev, n_classes))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)


def load_task(task):
    d = ARTIFACTS_DIR / task
    return (np.load(d/"X_train.npy"), np.load(d/"y_train.npy"),
            np.load(d/"X_val.npy"),   np.load(d/"y_val.npy"),
            np.load(d/"X_test.npy"),  np.load(d/"y_test.npy"),
            joblib.load(d/"preprocessor.pkl"))


def wilson_ci(n_correct, n_total, alpha=0.05):
    if n_total == 0:
        return 0.0, 1.0
    p = n_correct / n_total
    z = norm.ppf(1 - alpha / 2)
    denom = 1 + z * z / n_total
    center = (p + z * z / (2 * n_total)) / denom
    half = z * sqrt(p * (1 - p) / n_total
                    + z * z / (4 * n_total * n_total)) / denom
    return max(0.0, center - half), min(1.0, center + half)


def set_seed(seed):
    torch.manual_seed(seed)
    np.random.seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def train_one(task, seed=42, epochs=200, patience=15, lr=1e-3,
              batch_size=32, verbose=True):
    set_seed(seed)
    X_tr, y_tr, X_va, y_va, X_te, y_te, pre = load_task(task)
    n_classes = len(pre["encoder"].classes_)
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = MLP(X_tr.shape[1], n_classes).to(device)
    opt   = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()

    tr_ds = TensorDataset(torch.from_numpy(X_tr).float(),
                          torch.from_numpy(y_tr).long())
    tr_ld = DataLoader(tr_ds, batch_size=batch_size, shuffle=True)

```

```python
    X_va_t = torch.from_numpy(X_va).float().to(device)
    y_va_t = torch.from_numpy(y_va).long().to(device)

    best_val, best_state, bad = float("inf"), None, 0
    history = {"train_loss": [], "val_loss": [], "val_acc": []}

    for ep in range(epochs):
        model.train()
        total = 0.0
        for xb, yb in tr_ld:
            xb, yb = xb.to(device), yb.to(device)
            opt.zero_grad()
            logits = model(xb)
            loss = loss_fn(logits, yb)
            loss.backward(); opt.step()
            total += loss.item() * len(xb)
        tr_loss = total / len(tr_ds)

        model.eval()
        with torch.no_grad():
            val_logits = model(X_va_t)
            val_loss = loss_fn(val_logits, y_va_t).item()
            val_acc  = (val_logits.argmax(1) == y_va_t).float().mean().item()
        history["train_loss"].append(tr_loss)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        if val_loss < best_val - 1e-4:
            best_val, bad = val_loss, 0
            best_state = {k: v.clone() for k, v in model.state_dict().items()}
        else:
            bad += 1
            if bad >= patience:
                if verbose:
                    log.info(f"  Early stop at epoch {ep} "
                             f"(best val_loss={best_val:.4f})")
                break

    model.load_state_dict(best_state)
    model.eval()
    with torch.no_grad():
        X_te_t = torch.from_numpy(X_te).float().to(device)
        logits = model(X_te_t)
        y_pred = logits.argmax(1).cpu().numpy()
    n_correct = int((y_pred == y_te).sum())
    acc_te = n_correct / len(y_te) if len(y_te) else 0.0
    return model, history, acc_te, n_correct, len(y_te)
```

Listing 1: Top of `mlp_train.py` — imports, paths, logging, loader, Wilson CI, model.

```python
results = {}
for task in ["sf", "mod", "beacon", "pkt"]:
```

```
    log.info(f"--- Task: {task} ---")
    X_tr, y_tr, _, _, _, _, pre = load_task(task)
    n_classes = len(pre["encoder"].classes_)
    log.info(f"  Train shape={X_tr.shape}, classes={n_classes}, "
             f"features={pre['feature_cols']}")
    if n_classes < 2:
        log.warning(f"  {task}: only {n_classes} class --- "
                    f"skipping MLP (cross-entropy degenerate for one class)")
        results[task] = {"skipped": True, "reason": "single class in train"}
        continue

    t0 = time.time()
    model, history, acc_te, n_correct, n_total = train_one(task, seed=42)
    elapsed = time.time() - t0
    lo, hi = wilson_ci(n_correct, n_total)
    log.info(f"  MLP test acc = {acc_te:.3f}  [95% CI {lo:.3f}, {hi:.3f}]"
             f"({n_correct}/{n_total})  trained in {elapsed:.1f}s")

    torch.save({"state_dict": model.state_dict(),
                "n_features": X_tr.shape[1], "n_classes": n_classes,
                "seed": 42, "test_acc": acc_te, "ci": [lo, hi],
                "elapsed_s": elapsed, "history": history},
               ARTIFACTS_DIR / task / "mlp.pt")

    results[task] = {
        "test_acc": acc_te, "ci_lo": lo, "ci_hi": hi,
        "n_correct": n_correct, "n_total": n_total,
        "elapsed_s": elapsed, "n_features": X_tr.shape[1],
        "n_classes": n_classes, "feature_cols": pre["feature_cols"],
    }
```

Listing 2: Canonical training run (seed 42) per task with single-class skip and Wilson CI.

4.2 Step 2 — Learning curves

Plot training and validation loss as functions of epoch. The two should track each other; validation loss rising while training loss falls is the canonical overfitting signature.

```
for task, res in results.items():
    if res.get("skipped"):
        continue
    h = torch.load(ARTIFACTS_DIR / task / "mlp.pt",
                   weights_only=False)["history"]
    fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
    ax[0].plot(h["train_loss"], label="train")
    ax[0].plot(h["val_loss"],   label="val")
    ax[0].set_xlabel("Epoch"); ax[0].set_ylabel("Loss"); ax[0].legend()
    ax[0].set_title(f"{task} --- loss curves")
    ax[1].plot(h["val_acc"])
    ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("Val accuracy")
    ax[1].set_title(f"{task} --- val acc")
```

```
    fig.tight_layout()
    out = FIGS_DIR / f"mlp_curves_{task}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    log.info(f"  Saved {out}")
```

4.3 Step 3 — Probability calibration

A classifier that outputs a 90% probability for a class should be correct 90% of the time it does so. Plot a reliability diagram (binned empirical frequency vs. predicted probability).

```
def calibration_plot(model, X_te, y_te, task):
    model.eval()
    device = next(model.parameters()).device
    X_t = torch.from_numpy(X_te).float().to(device)
    with torch.no_grad():
        probs = torch.softmax(model(X_t), dim=1).cpu().numpy()
    pred     = probs.argmax(axis=1)
    max_prob = probs.max(axis=1)
    correct  = (pred == y_te).astype(int)
    # Uniform bins over [0, 1]; quantile bins waste resolution at the
    # high-confidence end where most predictions live.
    n_bins = 8
    frac_pos, mean_pred = calibration_curve(
        correct, max_prob, n_bins=n_bins, strategy="uniform")
    fig, ax = plt.subplots(figsize=(5, 4.5))
    ax.plot([0, 1], [0, 1], "--", color="gray", label="perfect")
    ax.plot(mean_pred, frac_pos, "o-", color="steelblue", label="MLP")
    ax.set_xlabel("Predicted confidence (top-1 prob)")
    ax.set_ylabel("Empirical accuracy")
    ax.set_title(f"Reliability diagram --- {task}")
    ax.set_xlim(0, 1); ax.set_ylim(0, 1); ax.legend(loc="lower right")
    fig.tight_layout()
    out = FIGS_DIR / f"mlp_reliability_{task}.png"
    fig.savefig(out, dpi=150)
    plt.close(fig)
    log.info(f"  Saved {out}")
    return list(zip(mean_pred.tolist(), frac_pos.tolist()))


for task, res in results.items():
    if res.get("skipped"):
        continue
    pkl = torch.load(ARTIFACTS_DIR / task / "mlp.pt", weights_only=False)
    _, _, _, _, X_te, y_te, _ = load_task(task)
    model = MLP(pkl["n_features"], pkl["n_classes"])
    model.load_state_dict(pkl["state_dict"])
    res["calibration"] = calibration_plot(model, X_te, y_te, task)
```

If the curve sits above the diagonal, the model is *under*-confident (real accuracy is higher than reported probability); if below, it is *over*-confident. For deployment, over-confidence is the dangerous case.

Why Calibration Matters for Spectrum Monitoring

The downstream consumer of this classifier's output is a decision process that may weight SF-probability estimates when doing multi-beacon tracking. An over-confident classifier will repeatedly assert wrong labels with high probability, pushing the tracking system toward false associations. A well-calibrated classifier — even one with slightly lower raw accuracy — produces more useful probability estimates for downstream fusion.

4.4 Step 4 — Random-seed robustness

Re-train the MLP for each non-skipped task with seeds 0, 1, 2, 3 (in addition to the canonical seed-42 run from Step 1). Report mean and standard deviation of the test accuracy per task. If the std exceeds ∼0.02 on any task, that model is unstable; consider more regularisation or additional data augmentation before promoting it. The Step 1 `mlp.pt` (seed 42) is left untouched — the seed sweep is for measurement, not replacement.

```
log.info("=== Seed robustness ===")
for task, res in results.items():
    if res.get("skipped"):
        continue
    accs = []
    for seed in range(4):
        _, _, acc, _, _ = train_one(task, seed=seed, verbose=False)
        accs.append(acc)
    mean = float(np.mean(accs))
    std  = float(np.std(accs))
    log.info(f"  {task:<8} seeds 0-3: accs={[round(a, 3) for a in accs]}"
             f"mean={mean:.3f} std={std:.3f}")
    if std > 0.02:
        log.warning(f"  {task}: seed std {std:.3f} > 0.02 --- "
                    f"model is unstable; consider more regularisation")
    res["seed_sweep"] = {"seeds": list(range(4)), "accs": accs,
                         "mean": mean, "std": std}
```

4.5 Step 5 — MLP vs. classical benchmarks

Read the RF and SVM test accuracies from the LH4-C/D manifests and print a head-to-head summary to the log. The script also writes `mlp_manifest.json` so LH4-F and LH4-H can read these numbers without re-training.

```
def load_manifest(name):
    p = SCRIPT_DIR / name
    if not p.exists():
        log.warning(f"{name} not found --- comparison column will be n/a")
        return {}
    with open(p) as f:
        return json.load(f).get("results", {})

rf_manifest  = load_manifest("baselines_manifest.json")
svm_manifest = load_manifest("svm_manifest.json")

```

```python
log.info("=== RF vs SVM vs MLP Summary ===")
log.info(f"{'Task':<8} {'RF':>8} {'SVM':>8} {'MLP':>8} {'MLP CI':>20} "
         f"{'seed std':>10}")
for task in ["sf", "mod", "beacon", "pkt"]:
    res = results[task]
    if res.get("skipped"):
        log.info(f"{task:<8} (skipped)  reason: {res['reason']}")
        continue
    rf  = (rf_manifest.get(task,  {}) or {}).get("rf",  {}).get("acc")
    svm = (svm_manifest.get(task, {}) or {}).get("test_acc")
    rf_s  = f"{rf:>8.3f}"  if rf  is not None else f"{'n/a':>8}"
    svm_s = f"{svm:>8.3f}" if svm is not None else f"{'n/a':>8}"
    sweep = res.get("seed_sweep", {})
    std_s = f"{sweep.get('std', 0):>10.3f}" if sweep else f"{'n/a':>10}"
    log.info(f"{task:<8} {rf_s} {svm_s} {res['test_acc']:>8.3f} "
             f"   [{res['ci_lo']:.3f}, {res['ci_hi']:.3f}] {std_s}")

manifest = {
    "generated_at": datetime.now(timezone.utc).isoformat(),
    "run_stamp":    run_stamp,
    "log_archive":  str(log_archive),
    "device":       "cuda" if torch.cuda.is_available() else "cpu",
    "results":      results,
}
with open(MANIFEST_PATH, "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2, default=str)
log.info(f"Wrote {MANIFEST_PATH}")
log.info("=== mlp_train.py done ===")
```

Task	RF	SVM	MLP	Seed std	Notes
SF Detection					
Modulation	(skipped — only LoRa collected)				
Emitter ID					
Packet Type					

Table 2: Three-way comparison. The MLP is only "worth it" for a task where its test accuracy clearly exceeds both classical benchmarks and its reliability curve is at least as good as the SVM's. Fill from the script's printed summary or from `mlp_manifest.json`.

5 Verification Checklist

- ☐ MLP trained to completion (or early stop) on every task with ≥ 2 classes (`sf`, `beacon`, `pkt`; `mod` skipped with logged reason).
- ☐ Wilson 95% CI reported for each MLP test accuracy.
- ☐ Loss curves show no pathological gap between train and val loss.

- ☐ Reliability diagrams generated; any systematic miscalibration noted.
- ☐ Seed-robustness: 4 independent runs per task (seeds 0–3) with std reported; std ≤ 0.02 unless flagged.
- ☐ `Code/AIML/artifacts/<task>/mlp.pt` saved for each non-skipped task (canonical seed 42).
- ☐ `Code/AIML/mlp_manifest.json` written.
- ☐ `Code/AIML/Logs/mlp_train_<timestamp>.log` archived.

Lab Handout 4-E Deliverable

1. `Code/AIML/mlp_train.py`: the runnable script.
2. `Code/AIML/mlp_manifest.json`: per-task accuracy with Wilson CI, seed-sweep mean/std, calibration bins, training history.
3. `Code/AIML/artifacts/{sf,beacon,pkt}/mlp.pt`: saved state dict (seed 42), training history, test accuracy.
4. `Code/AIML/figs/mlp_curves_<task>.png`: loss + val-acc curves per non-skipped task.
5. `Code/AIML/figs/mlp_reliability_<task>.png`: reliability diagram per non-skipped task.
6. Filled three-way benchmark table with winner column and 2–3 sentence commentary on why the MLP does or does not surpass the classical benchmarks.
7. Seed-robustness: mean $\pm$ std test accuracy over 4 runs per task, with any std > 0.02 flagged.
8. `Code/AIML/Logs/mlp_train_<timestamp>.log`: archived audit trail.